



FACULTY OF SCIENCE AND TECHNOLOGY

PHP CRUD Practical Exam

AI-Assisted Web Programming • Plain PHP + MySQL

Duration	2 hours	Total Marks	100
Frameworks	Not allowed	External Libraries	Not allowed
AI Tools	Allowed	Database	MySQL / MariaDB

Student Name: Su Man Student ID: 202300232

1. Exam Task

Build a complete Student Task Manager web application using **plain PHP, MySQL/MariaDB, HTML, and CSS only**. The application must allow a user to create, view, update, and delete academic tasks. You may use an AI assistant during the exam, but you remain responsible for the correctness and understanding of all submitted code.

2. Required Database

Create a database table named **tasks** with at least the following fields:

Field	Suggested Type	Purpose
id	INT, PK, AUTO INCREMENT	Unique task ID
title	VARCHAR(150)	Task title
description	TEXT	Task details
category	VARCHAR(50)	Task category
priority	VARCHAR(20)	Priority level
due_date	DATE	Due date
completed	TINYINT(1)	Completion status
created_at	DATETIME / TIMESTAMP	Creation date/time

3. Required CRUD Features

- **CREATE:** Add a new task using an HTML form and POST.
- **READ:** Display all tasks retrieved from the database.
- **UPDATE:** Edit an existing task and save the changes.
- **DELETE:** Delete an existing task from the database.

A suggested file structure is shown below. You may organize your files differently if the required functionality is present.

index.php - list/read tasks
create.php - add a task

edit.php - update a task
 delete.php - delete a task
 db.php - database connection
 functions.php - reusable functions (if used)
 style.css - optional custom styling

4. PHP Requirements

Your solution must demonstrate the following concepts where appropriate:

- Variables and appropriate PHP data types
- Arrays
- Conditional statements (if / else)
- Loops
- At least one reusable function written by you
- include or require
- GET and POST request handling
- HTML forms
- Server-side input validation
- Database connection using PDO
- SQL SELECT, INSERT, UPDATE, and DELETE
- Prepared statements for queries containing user input
- At least one use of a PHP session (for example, a success/error flash message)
- Safe output using htmlspecialchars() where user-entered text is displayed

5. Assigned Challenge

Your instructor will assign ONE additional challenge. Implement it in your application. Examples include filtering incomplete tasks, highlighting overdue tasks, sorting by due date, counting completed tasks, filtering by priority, searching by title, validating due dates, or displaying a category count.

My assigned challenge: Highlight overdue tasks any task that is not completed and whose due date is before today is displayed with a red background.

6. Rules for AI-Assisted Development

1. AI assistants such as ChatGPT, Claude, Gemini, or Copilot may be used.
2. You may ask AI for explanations, debugging help, code suggestions, SQL, or examples.
3. You must personally assemble, test, and submit the final application.
4. You are responsible for every line of submitted code, including AI-generated code.
5. After submission, you will complete an individual Code Defense and Live Code Modification. During this verification stage, AI tools, internet searches, notes, and help from other students are not permitted.
6. Frameworks and external libraries/packages are not permitted. Examples: Laravel, Symfony, CodeIgniter, Bootstrap, jQuery, Composer packages, or similar third-party code.
7. Do not copy another student's project or share your completed project with another student.

IMPORTANT: AI is permitted, but understanding is part of the assessment. A working application that you cannot explain may lose marks during verification.

7. Suggested Time Management

Time	Recommended Activity
0-15 min	Read requirements, create database/table, plan files.
15-90 min	Implement CRUD and required PHP functionality.
90-110 min	Test, debug, validate inputs, and complete assigned challenge.
110-120 min	Prepare submission, export SQL, complete AI-use reflection.

8. Submission Requirements

- Complete project folder containing all PHP, HTML/CSS, and related files.
- SQL export (.sql) containing the database/table structure required to run the application.
- README.md or README.txt containing your name, student ID, setup instructions, and AI-use reflection.
- The application must run on the exam environment before submission.

9. AI-Use Reflection

Include the following in your README file:

Student Name: Su Man
Student ID: 202300232

AI tool(s) used:

Three examples of how AI helped me:

1. Generated the initial structure of all files (db.php, functions.php, index.php, create.php, edit.php, delete.php, style.css) with PDO, prepared statements, and server-side validation already in place.
2. Explained the key concepts in simple terms (prepared statements vs SQL injection, htmlspecialchars vs XSS, GET vs POST, why exit is needed after header, how flash sessions work), which helped me prepare for the code defense.
3. Helped me implement and test my assigned challenge (highlighting overdue tasks in red using a CSS class) and fixed bugs like deleting JavaScript that was not allowed and restoring PDO.

One AI-generated suggestion or piece of code that I changed or rejected:

What was it? AI first suggested adding a JavaScript confirm dialog on the Delete button (onclick="return confirm('Delete this task?')

Why did I change/reject it? The exam does not allow JavaScript, so I removed the confirm dialog and used a plain POST form for delete. I also reverted MySQLi back to PDO because the rubric explicitly requires PDO with prepared statements, try/catch, and `ERRMODE_EXCEPTION`.

The part of this application I understand least:

How prepared statements with placeholders (? and named :params) prevent SQL injection internally, and exactly what happens step by step when `header("Location: ...")` is called before `exit`.

The part of this application I understand least: How prepared statements with placeholders prevent SQL injection internally, and what exactly happens when `header("Location: ...")` runs before `exit`.

10. Grading Rubric

Criterion	Marks	What is assessed
Working CRUD application	15	Create, read, update, and delete operate correctly.
PHP fundamentals & database	10	PHP basics, PDO, SQL, GET/POST, functions, sessions, and prepared statements.
Validation, security & organization	10	Server-side validation, safe output, sensible structure, and readable code.
Assigned individual challenge	10	The assigned feature is implemented and works correctly.
AI-use reflection	5	Clear record of how AI was used, evaluated, changed, or rejected.
CODE EXPLANATION / DEFENSE	25	Without AI: explain selected code, data flow, PHP/SQL behavior, decisions, and consequences of changes.
LIVE CODE MODIFICATION	25	Without AI: implement, test, and explain an instructor-requested change to the submitted application.
TOTAL	100	

10.1 Code Explanation / Defense - 25 Marks

The instructor may select any part of your submitted project. You must explain it in your own words without AI assistance. Questions may require you to trace a form submission from HTML to PHP and MySQL, explain a prepared statement, distinguish GET from POST, identify validation or security measures, explain a condition or loop, or predict what will happen if selected code is changed or removed.

Level	Marks	Performance
Excellent	21-25	Explains code accurately and independently; traces data/control flow; understands PHP/SQL decisions; answers follow-up questions clearly.
Good	16-20	Explains most code correctly with minor gaps; understands the main flow and purpose of important statements.
Developing	9-15	Understands basic purpose but struggles with details, interactions, or consequences of changes.
Limited	1-8	Can identify only small parts of the code; explanations show substantial dependence on generated code.
No evidence	0	Cannot meaningfully explain the submitted code.

10.2 Live Code Modification - 25 Marks

The instructor will give you a small change after submission. The exact modification will not be announced in advance. You must make the change yourself, test it, and explain what you changed. No AI assistance is permitted during this stage. Possible tasks include adding a filter, changing sort order, adding a counter, changing validation, displaying overdue tasks, or making a similar modification to the submitted application.

Level	Marks	Performance
-------	-------	-------------

Excellent	21-25	Locates the correct code, implements the requested change independently, tests it successfully, and explains the modification.
Good	16-20	Completes most or all of the change with minor errors or limited instructor prompting and explains the approach.
Developing	9-15	Makes partial progress but needs significant prompting or has difficulty integrating/testing the change.
Limited	1-8	Can make only very small changes and cannot independently locate or adapt the relevant code.
No evidence	0	Unable to make a meaningful modification.

Verification rule: The Code Defense and Live Code Modification are individual assessments. AI tools, internet searches, notes, and assistance from other students are not allowed.

11. Final Checklist

- ☐ My application runs without fatal PHP errors.
- ☐ All four CRUD operations work.
- ☐ I used PDO and prepared statements.
- ☐ I validated required input on the server side.
- ☐ I escaped user-entered output where appropriate.
- ☐ My assigned challenge works.
- ☐ My SQL export is included.
- ☐ My README and AI-use reflection are included.
- ☐ I can explain the code I submitted and modify it independently without AI.

Submit before the end of the 2-hour examination period.